

TheKnife

Diagrammi ER e UML

Schema ER Concettuale | Schema ER Ristrutturato | Diagramma delle Classi | Diagrammi di Sequenza

Laboratorio Interdisciplinare B - a.a. 2025/2026 | Università degli Studi dell'Insubria

Matteo Vigano 760537 | Fabio Vecaj 761232 | De Zuane Samuele 763267 - sede CO

1. Schema ER Concettuale

Lo schema ER concettuale modella il dominio applicativo di TheKnife indipendentemente dall'implementazione fisica. Identifica entità, associazioni, cardinalità e vincoli di identificazione.

1.1 Entità

Il modello comprende quattro entità. UTENTE e RISTORANTE sono entità forti con identificatore proprio. RECENSIONE e PREFERITO sono entità deboli il cui identificatore dipende dalle entità partecipanti.

UTENTE	
PK	username
	nome
	cognome
	password_hash
	data_nascita (opz.)
	domicilio (opz.)
	ruolo { cliente ristoratore }

RISTORANTE	
PK	nome
	nazione
	città
	indirizzo (opz.)
	latitudine
	longitudine
	fascia_prezzo
	delivery { sì no }
	prenotazione { sì no }
	tipo_cucina

<<entità debole>> RECENSIONE	
PK	(nome_ristorante, username_cliente)
	stelle { 1..5 }
	testo
	risposta (opz.)

<<entità debole>> PREFERITO	
PK	(username, nome_ristorante)
	-- nessun attributo proprio --

1.2 Associazioni e cardinalità

Associazione	Entità 1	Card. min/max 1	Entità 2	Card. min/max 2	Tipo
GESTISCE	UTENTE (ristoratore)	(1,1)	RISTORANTE	(0,N)	1:N
SCRIVE	UTENTE (cliente)	(0,N)	RECENSIONE	(1,1)	1:N parziale
RICEVE	RISTORANTE	(0,N)	RECENSIONE	(1,1)	1:N parziale
SALVA	UTENTE (cliente)	(0,N)	RISTORANTE	(0,N)	N:M

1.3 Descrizione delle associazioni

GESTISCE (UTENTE -> RISTORANTE) : 1:N

Un ristorante possiede 0..N ristoranti. Ogni ristorante ha esattamente 1 proprietario.

SCRIVE (UTENTE -> RECENSIONE) : 1:N

Un cliente scrive 0..N recensioni. Ogni recensione è scritta da esattamente 1 cliente.

Vincolo: al massimo una recensione per coppia (cliente, ristorante).

RICEVE (RISTORANTE -> RECENSIONE) : 1:N

Un ristorante riceve 0..N recensioni. Ogni recensione si riferisce a esattamente 1 ristorante.

SALVA (UTENTE <-> RISTORANTE) : N:M

Un cliente salva 0..N ristoranti. Un ristorante può essere nei preferiti di 0..N clienti.

Si traduce in tabella di raccordo senza attributi propri.

1.4 Vincoli aggiuntivi (linguaggio naturale)

1. Un utente ha ruolo 'cliente' oppure 'ristoratore', mai entrambi.
2. Un cliente può scrivere al massimo 1 recensione per ristorante (PK composta).
3. Le stelle vanno da 1 a 5 inclusi (intero).
4. Solo ristoratori possono creare ristoranti; solo clienti possono scrivere recensioni e gestire preferiti.
5. Il ristorante risponde a una recensione con al massimo 1 testo (attributo nullable).
6. Latitudine: -90..90. Longitudine: -180..180. Fascia prezzo: valore positivo.
7. La password è memorizzata come hash SHA-256 (64 caratteri esadecimali).

1.5 Identificatori

Entita	Identificatore	Tipo	Note
UTENTE	username	Interno semplice	Univoco, immutabile
RISTORANTE	nome	Interno semplice	Univoco nella piattaforma
RECENSIONE	(username_cliente, nome_ristorante)	Esterno composto	Entita debole; dipende da UTENTE e RISTORANTE
PREFERITO	(username, nome_ristorante)	Esterno composto	Entita debole; dipende da UTENTE e RISTORANTE

2. Schema ER Ristrutturato

La ristrutturazione prepara lo schema alla traduzione relazionale. Le operazioni effettuate riguardano entità deboli, gestione della generalizzazione e scelte sugli attributi.

2.1 Decisioni di ristrutturazione

Elemento	Operazione effettuata	Motivazione
Entità debole RECENSIONE	Tradotta in tabella con PK composta (nome_ristorante, username_cliente); FK verso ristoranti e utenti con ON DELETE CASCADE	Dipende da due entità forti; la coppia identifica univocamente ogni recensione
Entità debole PREFERITO	Tradotta in tabella di raccordo con PK composta (username, nome_ristorante); FK verso utenti e ristoranti	Realizza la relazione N:M SALVA senza attributi propri
Attributo 'risposta'	Mantenuto come TEXT nullable in RECENSIONI	Molteplicità 0..1: evita tabella separata; al massimo 1 risposta per recensione
Generalizzazione ruolo	NON separata in tabelle CLIENTE/RISTORATORE. Mantenuto CHECK constraint su 'ruolo'	Semplifica il modello; le differenze sono gestite a livello applicativo
Associazione GESTISCE	FK 'proprietario' in RISTORANTI -> utenti(username)	Cardinalità 1:N; la FK va nella tabella lato N
Associazioni SCRIVE+RICEVE	FK nella tabella RECENSIONI: username_cliente e nome_ristorante	Costituiscono la PK composta dell'entità debole
Associazione SALVA	Tabella raccordo PREFERITI con due FK	Pattern standard per N:M nel modello relazionale
Attributi calcolati	mediaStelle e numeroRecensioni NON memorizzati: calcolati con AVG()+COUNT()	Evita ridondanza e inconsistenza nei dati

2.2 Schema relazionale (notazione testuale)

```
utenti (  
    username          VARCHAR(50)    PRIMARY KEY,  
    nome              VARCHAR(50)    NOT NULL,  
    cognome           VARCHAR(50)    NOT NULL,  
    password_hash     CHAR(64)       NOT NULL,  
    data_nascita      DATE,  
    domicilio         VARCHAR(100),  
    ruolo             VARCHAR(20)    NOT NULL  
                                CHECK (ruolo IN ('cliente','ristoratore'))  
);  
  
ristoranti (  
    nome              VARCHAR(100)    PRIMARY KEY,  
    nazione           VARCHAR(50)    NOT NULL,  
    citta             VARCHAR(100)    NOT NULL,  
    indirizzo         VARCHAR(200),  
    latitudine        FLOAT          CHECK (latitudine BETWEEN -90 AND 90),  
    longitudine       FLOAT          CHECK (longitudine BETWEEN -180 AND 180),  
    fascia_prezzo     NUMERIC(8,2)    NOT NULL CHECK (fascia_prezzo > 0),  
    delivery          BOOLEAN        DEFAULT FALSE,  
    prenotazione      BOOLEAN        DEFAULT FALSE,  
    tipo_cucina       VARCHAR(100)    NOT NULL,  
    proprietario      VARCHAR(50)    REFERENCES utenti(username) ON DELETE CASCADE  
);  
  
recensioni (  
    nome_ristorante   VARCHAR(100)    REFERENCES ristoranti(nome) ON DELETE CASCADE,  
    username_cliente  VARCHAR(50)    REFERENCES utenti(username) ON DELETE CASCADE,  
    stelle            SMALLINT        NOT NULL CHECK (stelle BETWEEN 1 AND 5),  
    testo             TEXT            NOT NULL,  
    risposta          TEXT,  
    PRIMARY KEY (nome_ristorante, username_cliente)  
);  
  
preferiti (  
    username          VARCHAR(50)    REFERENCES utenti(username) ON DELETE CASCADE,  
    nome_ristorante   VARCHAR(100)    REFERENCES ristoranti(nome) ON DELETE CASCADE,  
    PRIMARY KEY (username, nome_ristorante)  
);
```

2.3 Tabelle dettagliate con vincoli

UTENTI

Colonna	Tipo	Vincolo	Descrizione
username	VARCHAR(50)	PRIMARY KEY	ID univoco utente
nome	VARCHAR(50)	NOT NULL	Nome
cognome	VARCHAR(50)	NOT NULL	Cognome
password_hash	CHAR(64)	NOT NULL	Hash SHA-256
data_nascita	DATE	nullable	Opzionale
domicilio	VARCHAR(100)	nullable	Citta residenza
ruolo	VARCHAR(20)	CHECK cliente ristoratore	Ruolo piattaforma

RISTORANTI

Colonna	Tipo	Vincolo	Descrizione
nome	VARCHAR(100)	PRIMARY KEY	Nome univoco
nazione	VARCHAR(50)	NOT NULL	
citta	VARCHAR(100)	NOT NULL	
indirizzo	VARCHAR(200)	nullable	
latitudine	FLOAT	CHECK -90..90	
longitudine	FLOAT	CHECK -180..180	
fascia_prezzo	NUMERIC(8,2)	NOT NULL CHECK >0	Euro
delivery	BOOLEAN	DEFAULT FALSE	
prenotazione	BOOLEAN	DEFAULT FALSE	
tipo_cucina	VARCHAR(100)	NOT NULL	
proprietario	VARCHAR(50)	FK->utenti CASCADE	Username ristoratore

RECENSIONI

Colonna	Tipo	Vincolo	Descrizione
nome_ristorante	VARCHAR(100)	PK + FK->ristoranti CASCADE	Parte PK composta
username_cliente	VARCHAR(50)	PK + FK->utenti CASCADE	Parte PK composta

stelle	SMALLINT	NOT NULL CHECK 1..5	Valutazione
testo	TEXT	NOT NULL	Testo recensione
risposta	TEXT	nullable	Risposta ristorante

PREFERITI

Colonna	Tipo	Vincolo	Descrizione
username	VARCHAR(50)	PK + FK->utenti CASCADE	Parte PK composta
nome_ristorante	VARCHAR(100)	PK + FK->ristoranti CASCADE	Parte PK composta

3. UML - Diagramma delle Classi

Il diagramma mostra la struttura statica del sistema nei tre moduli Maven: theknife-common, theknife-server e theknife-client.

3.1 Modulo theknife-common

Ristorante

<<Serializable>> Ristorante	
- nome : String {PK} - nazione : String - citta : String - indirizzo : String - latitudine : double - longitudine : double - fasciaPrezzo : double - delivery : boolean - prenotazione : boolean - tipoCucina : String - proprietario : String {FK->Utente} - mediaStelle : double {calcolato} - numeroRecensioni : int {calcolato}	
+ Ristorante() + Ristorante(nome,nazione,citta,ind,lat,lon,prezzo,del,pren,cucina) + getNome() : String + setNome(n:String) : void ... getter/setter per tutti gli attributi + isDelivery() : boolean + isPrenotazione() : boolean + toString() : String	

Utente e Recensione

<<Serializable>> Utente	<<Serializable>> Recensione
- username : String {PK} - nome : String - cognome : String - passwordHash : String - dataNascita : String - domicilio : String - ruolo : String + Utente() + Utente(username,nome,cognome,...) + isCliente() : boolean + isRistoratore() : boolean ... getter/setter + toString() : String	- nomeRistorante : String {FK} - usernameCliente : String {FK} - stelle : int {1..5} - testo : String - risposta : String {nullable} + Recensione() + Recensione(nomeRist,userCliente,stelle,testo) + hasRisposta() : boolean + getRisposta() : String + setRisposta(r:String) : void ... getter/setter

Request e Response (protocollo di comunicazione)

<<Serializable>> Request	<<Serializable>> Response
- comando : CommandType - usernameLoggato : String {nullable} - parametri : Map<String,Object> + Request(cmd:CommandType, user:String) + aggiungiParametro(k:String,v:Object) : Request [Fluent API - ritorna this] + getComando() : CommandType + getParametri() : Map<String,Object>	- successo : boolean - messaggio : String - dato : Object + ok(dato:Object) : Response {static} + errore(msg:String) : Response {static} + isSuccesso() : boolean + getMessaggio() : String + getDatoTipizzato() : T {unchecked}

CommandType (enumerazione)

Categoria	Valori
Guest	CERCA_RISTORANTI, VISUALIZZA_RECENSIONI
Autenticazione	LOGIN, REGISTRAZIONE
Cliente	CLIENTE_VISUALIZZA_PREFERITI, CLIENTE_AGGIUNGI_PREFERITO, CLIENTE_RIMUOVI_PREFERITO, CLIENTE_VISUALIZZA_MIE_RECENSIONI, CLIENTE_AGGIUNGI_RECENSIONE, CLIENTE_MODIFICA_RECENSIONE, CLIENTE_ELIMINA_RECENSIONE
Ristoratore	RISTORATORE_AGGIUNGI_RISTORANTE, RISTORATORE_VISUALIZZA_RIEPILOGO, RISTORATORE_VISUALIZZA_RECENSIONI, RISTORATORE_RISPONDI_RECENSIONE

3.2 Modulo theknife-server

ServerTK	<<Runnable>> ClientHandler	<<AutoCloseable>> DatabaseManager
<pre> - portaServer : int - pool : ExecutorService {50 thread} - dbConfig : DbConfig </pre>	<pre> - socket : Socket - utenteLoggato : Utente {sessione per- thread} - dbConfig : DbConfig </pre>	<pre> - conn : Connection {JDBC, una per thread} + login(user,hash) : Utente + registrazione(...) : boolean + cercaRistoranti(filtri) : List<Ristorante> + aggiungiPreferito(u,r) : boolean + rimuoviPreferito(u,r) : boolean + visualizzaPreferiti(u) : List<Ristorante> + aggiungiRecensione(...) : boolean + modificaRecensione(...) : boolean + eliminaRecensione(...) : boolean + visualizzaRecensioni(r) : List<Recensione> + rispondiRecensione(...) : boolean + riepilogoRistoratore(u) : List<Ristorante> + aggiungiRistorante(...) : boolean + close() : void </pre>
<pre> + main(args:String[]) : void {static} - avvia() : void </pre>	<pre> + run() : void - elabora(req:Request,db:DatabaseManager):Resp onse - checkCliente() : boolean - checkRistoratore() : boolean </pre>	

3.3 Modulo theknife-client

ClientTK

```
- utenteLoggato : Utente {static, nullable}
- connessione : ServerConnection {static}

+ main(args:String[]) : void {static}
+ isLoggato() : boolean {static}
+ getUtenteLoggato() : Utente {static}
+ setUtenteLoggato(u:Utente) : void {static}
+ logout() : void {static}
+ getConnessione() : ServerConnection {static}
```

ServerConnection

```
- socket : Socket
- in : ObjectInputStream
- out : ObjectOutputStream

+ ServerConnection(host:String, porta:int)
+ invia(req:Request) : Response
+ login(user,pass) : Response
+ registrazione(...) : Response
+ sha256(password:String) : String {static}
+ chiudi() : void
```

<<extends JFrame>>

FancyFrame

```
- cardLayout : CardLayout
- CARD_HOME, CARD_SEARCH, CARD_DETAIL, CARD_PREFERITI, CARD_RECENSIONI, CARD_RISTORANTI : String {static}
- homePanel : HomePanel
- searchPanel : SearchPanel
- detailPanel : RestaurantDetailPanel
- preferitiPanel : PreferitiPanel
- recensioniPanel : RecensioniPanel
- ristorantiPanel : RistorantiPanel

+ FancyFrame()
+ showCard(name:String) : void
+ getDetailPanel() : RestaurantDetailPanel
+ getSearchPanel() : SearchPanel
- buildTopBar() : JPanel
- buildSidebar() : JPanel
- aggiornaSidebar() : void
```

Pannelli GUI (tutti extends GradientPanel extends JPanel)

Pannello	Classe	Funzionalità principali
Home	HomePanel	Hero section, ricerca città/domicilio, 3 feature card cliccabili, CTA ristoratori
Esplora	SearchPanel	Ricerca con filtri, chip attivi rimovibili, griglia card, FiltriDialog
Dettaglio	RestaurantDetailPanel	Info ristorante, stelle Java2D, badge servizi, lista recensioni con risposta
Preferiti	PreferitiPanel	Griglia preferiti, rimozione rapida X, ordinamento stelle/alfabetico
Recensioni	RecensioniPanel	Filtro stelle Java2D, CRUD recensioni via dialog, risposta ristoratore
Miei locali	RistorantiPanel	Griglia locali, statistiche media+conteggio, dialog aggiunta ristorante

3.4 Relazioni tra classi

Classe origine	Classe dest.	Tipo	Molt.	Descrizione
ServerTK	ClientHandler	Crea	1->0..*	Istanza un ClientHandler per ogni client TCP accettato
ClientHandler	DatabaseManager	Ha-un	1->1	Una connessione JDBC per thread; try-with-resources
ClientHandler	Utente	Associazione	1->0..1	Mantiene la sessione utente per la durata della connessione
ClientTK	ServerConnection	Associazione	1->1	Connessione TCP persistente per tutta la sessione
FancyFrame	*Panel (6)	Composizione	1->1each	Contiene tutti i pannelli tramite CardLayout
*Panel	ClientTK	Dipendenza	*->1	Accedono a sessione e connessione via metodi statici
server	common	Dipendenza Maven	1->1	Usa Request, Response, Utente, Ristorante, Recensione
client	common	Dipendenza Maven	1->1	Usa le stesse classi serializzabili

4. UML - Diagrammi di Sequenza

I diagrammi documentano l'interazione temporale tra i componenti del sistema per i casi d'uso principali.

Legenda:
[TCP->] = Request serializzata su ObjectOutputStream verso il server
[TCP<-] = Response deserializzata da ObjectInputStream
testo = passo critico o messaggio di ritorno principale
SwingWorker = thread asincrono che non blocca l'EDT di Swing

4.1 Login utente

Pre: utente non loggato. Post: utenteLoggato in ClientTK, topbar aggiornata.

#	Utente	LoginDialog	SwingWorker	ClientHandler	DatabaseManager + PG
1	Digita username e password ; clicca Accedi				
2		Valida campi; sha256(password)->hash			
3		Crea Request(LOGIN){username,hash} [TCP->]	invia(req)		
4				switch(LOGIN)->db.login(u,h)	
5					SELECT FROM utenti WHERE username=? AND password_hash=?
6					ResultSet -> Utente
7				utenteLoggato=utente; Response.ok(utente)	
8			[TCP<-] Response.ok(utente)		
9		ClientTK.setUtenteLoggato(u)			
10		FancyFrame.aggiornaSidebar(); topbar aggiornata			

Percorso alternativo (credenziali errate):

Passo 6: ResultSet vuoto -> null

Passo 7: Response.errore("Username o password non corretti")

GUI mostra errore; utenteLoggato rimane null

4.2 Ricerca ristoranti con filtri

Pre: nessuna (disponibile anche a guest). Post: griglia risultati aggiornata.

#	Uten te	Search Panel	SwingWorker	ClientHandl er	DatabaseManag er + PG
1	Inseri sce citta; impo sta filtri; clicca Cerc a				
2		Mostra loader; lancia SwingW orker			
3			doInBackground(): Request(CERCA_RISTORANTI){citta,cucina,prezzoMin,pre zzoMax,stelle,delivery,prenotazione} [TCP->]		
4				switch(CERC A) -> db.cercaRisto ranti(filtri)	
5					SELECT r.*,AVG(rec.stelle) ,COUNT(*) FROM ristoranti r LEFT JOIN recensioni rec ON r.nome=rec.nome _ristorante WHERE r.citta=? [+filtri] GROUP BY r.nome
6					List<Ristorante> con mediaStelle e numeroRecensi oni
7			[TCP<-] Response.ok(lista)		
8			done() sull EDT: ordina per stelle DESC		
9		Aggiorn a griglia; badge Top rated al			

		primo se stelle>= 4.5			
--	--	-----------------------------	--	--	--

Note: PreparedStatement con WHERE clause dinamica previene SQL injection.
Se lista vuota: GUI mostra 'Nessun ristorante trovato a [citta]'.

4.3 Inserimento recensione (cliente)

Pre: cliente loggato. Post: recensione nel DB, lista aggiornata.

#	Client e	RestaurantDetailPanel	ServerConne ction	ClientHa ndler	DatabaseManager + PG
1	Apri detta glio; clicca 'Scrivi una recen sione'				
2		Apri RecensioneDialog: slider stelle + campo testo			
3	Impos ta 4 stelle; scrive testo; clicca Pubbl ica				
4		Crea Request(CLIENTE_AGGIUNGI_RECENSI ONE,username){nomeRist,stelle,testo} [TCP->]	invia(req)		
5				Verifica utenteLog gato!=null && isCliente()	
6					INSERT INTO recensioni(nome_ristorante,us ername_cliente,stelle,testo) VALUES(?,?,?,?)
7					Vincolo PK: se già esiste -> eccezione SQL catturata
8			[TCP<-] Response.ok("Recensione inserita")		
9		Mostra OK; ricarica lista recensioni [VISUALIZZA_RECENSIONI]			

Percorso alternativo (gia recensito):

Passo 7: violazione PK (nome_ristorante, username_cliente) -> eccezione catturata

Response.errore("Hai gia recensito questo ristorante")

L utente puo usare 'Modifica' per aggiornare la recensione esistente

4.4 Risposta del ristorante

Pre: ristorante loggato, almeno 1 recensione ricevuta. Post: campo risposta aggiornato.

#	Ristoratore	RecensioniPanel	ServerConnection	ClientHandler	DatabaseManager + PG
1	Apri Recensioni; vede lista recensioni ricevute				
2	Clicca Rispondi				
3		Mostra combobox recensioni; utente seleziona e scrive risposta			
4	Clicca Pubblica risposta				
5		Crea Request(RISTORATORE_RISPONDI_RECENSIONE,username){nomeRist,userCliente,risposta} [TCP->]	invia(req)		
6				Verifica isRistoratore(); verifica proprieta ristorante	
7					UPDATE recensioni SET risposta=? WHERE nome_ristorante=? AND username_cliente=?
8			[TCP<-] Response.ok("Risposta pubblicata")		
9		Aggiorna card: mostra riquadro blu 'Risposta del ristorante'			

Nota: una seconda risposta alla stessa recensione esegue UPDATE (sovrascrittura), non INSERT.
Il sistema ammette al massimo 1 risposta per recensione.

4.5 Aggiunta nuovo ristorante

Pre: ristorante loggato. Post: ristorante nel DB, griglia locali aggiornata.

#	Ristoratore	RistorantiPanel	ServerConnection	ClientHandler	DatabaseManager + PG
1	Clicca '+ Aggiungi ristorante'				
2		Apri NuovoRistoranteDialog con form			
3	Compila tutti i campi; clicca Salva				
4		Valida campi obbligatori e formato numerico (lat,lon,prezzo)			
5		Crea Request(RISTORATORE_AGGIUNGI_RISTORANTE,username){nome,citta,nazione,...} [TCP->]	invia(req)		
6				Verifica isRistoratore()	
7					INSERT INTO ristoranti(nome,nazione,citta,...,proprietario) VALUES(?,?,?,...,?)
8			[TCP<-] Response.ok("Ristorante aggiunto")		
9		Chiama refreshData(); griglia locali aggiornata			

Pattern comune a tutti i diagrammi di sequenza:

1. Validazione lato client (campi obbligatori, formati numerici)
2. Costruzione Request con CommandType specifico e Map parametri
3. Invio asincrono tramite SwingWorker (non blocca EDT)
4. Verifica autenticazione e ruolo lato server (ClientHandler)
5. Query JDBC con PreparedStatement (prevenzione SQL injection)
6. Costruzione Response e serializzazione su socket TCP
7. Aggiornamento GUI sull EDT nel metodo done() di SwingWorker